

■ Git Cheatsheet

Push · Rebase · Rename — the commands you always look up

■ 1. Push Rejected? (Remote is ahead of local)

This happens when someone else pushed to the remote `main` after your last pull. Git won't let you overwrite their work.

STEP 1 Fetch — see what's on the remote (no changes applied yet)

```
git fetch origin
git log --oneline HEAD..origin/main # see commits you're missing
```

STEP 2 Rebase — replay your commits on top of the remote

```
git pull --rebase origin main
```

STEP 3 Push — now the history is linear, push succeeds

```
git push origin main
```

■ Why rebase instead of merge? Rebase keeps history linear — no extra "Merge branch main" commit clutter. Use merge only when you want to preserve a feature-branch divergence.

⇒ ■ 2. Rename GitHub Repo + Update Local Remote

STEP 1 Rename on GitHub (web UI)

- ① Go to your repo page on github.com
- ② Click `Settings` (top-right tab)
- ③ Under `Repository name` → type the new name
- ④ Click `Rename` to confirm

STEP 2 Update local remote URL (terminal)

```
git remote -v # check old URL
git remote set-url origin https://github.com//.git
git remote -v # verify
git fetch origin # test it works
```

■ Renaming a local folder never affects git. Git only cares about the remote URL stored in `.git/config`. Renaming the folder on disk = harmless.

■ 3. Verify with a Test Commit

After a rename or rebase, confirm the remote connection works without touching any files:

```
git commit --allow-empty -m "Test: verify push" # no file changes needed
git push origin main
```

■ Daily Flow (Safe Default)

```
git status # see what's changed

git add # stage specific files

git commit -m "clear message" # commit

git pull --rebase origin main # sync with remote

git push origin main # publish
```

■ Quick Reference

Situation	Command
Remote ahead, want linear history	<code>git pull --rebase origin main</code>
Remote ahead, want a merge commit	<code>git pull origin main</code>
Fix conflict during rebase	<code>edit</code> → <code>git add</code> → <code>git rebase --continue</code>
Abort a bad rebase	<code>git rebase --abort</code>
Undo last commit, keep changes staged	<code>git reset --soft HEAD~1</code>
Undo last commit, discard all changes	<code>git reset --hard HEAD~1</code>
Force push safely (after rebase)	<code>git push --force-with-lease origin main</code>
See diverging commits	<code>git log --oneline HEAD..origin/main</code>
Change remote URL	<code>git remote set-url origin</code>

■ Common Errors → Fix

Error Message	Fix
<code>! [rejected] main -> main (fetch first)</code>	<code>git pull --rebase origin main</code>
<code>LF will be replaced by CRLF</code>	Harmless warning (Windows line-ends)
<code>fatal: not a git repository</code>	<code>git init</code> or <code>cd</code> into the repo
<code>Updates were rejected because remote contains work</code>	Pull first, then push
<code>error: failed to push some refs</code>	<code>git fetch origin</code> then <code>rebase/merge</code>
<code>CONFLICT (content): Merge conflict in</code>	Open file, fix markers, <code>git add</code> , <code>rebase --continue</code>

■■ Conflict During Rebase — Step-by-Step

```
git pull --rebase origin main

# CONFLICT (content): Merge conflict in app.py ← git pauses here

# 1. Open the file and look for conflict markers:

# <<<<<<< HEAD ← your local changes

# your code here

# =====

# remote code here

# >>>>>>> origin/main ← incoming remote changes

# 2. Edit the file to keep what you want, remove the markers
```

```
git add app.py # 3. Mark resolved
git rebase --continue # 4. Continue rebasing

# If things go wrong at any point:
git rebase --abort # undo everything, back to before rebase
```

■ Key Terms

Term	Meaning
origin	The default name for your remote repository (on GitHub/GitLab).
HEAD	Pointer to your current commit — the tip of your working branch.
rebase	Moves/replays your commits on top of another branch's tip.
fetch	Downloads remote changes but does NOT apply them locally.
--force-with-lease	Safer force-push: fails if someone else pushed since your last fetch.
HEAD~1	One commit before HEAD. HEAD~2 = two commits back, etc.

Git Cheatsheet • Push • Rebase • Rename • All commands tested on Git 2.x